Software Engineering

CSE Semester 5 — Academic Notes

Synthetic academic knowledge-base document created for CampusAI RAG evaluation and demonstration.

Unit 1 — Software Process and Requirements

- Software engineering applies systematic engineering principles to the development, operation, maintenance, and evolution of software.
- Software process models include waterfall, incremental development, prototyping, spiral development, and agile approaches.
- Functional requirements describe what a system should do. Non-functional requirements describe qualities or constraints such as performance, security, usability, and reliability.
- Requirements engineering includes elicitation, analysis, specification, validation, and management.
- A Software Requirements Specification (SRS) documents agreed requirements and provides a basis for design, development, and testing.

Unit 2 — Agile and Scrum

- Agile development emphasizes iterative delivery, customer collaboration, responding to change, and frequent feedback.
- Scrum uses roles such as Product Owner, Scrum Master, and Developers.
- Scrum artifacts include the Product Backlog, Sprint Backlog, and Increment.
- A Sprint is a time-boxed development cycle. Common events include Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective.
- User stories express requirements from a user's perspective, often using the form: As a [user], I want [goal], so that [benefit].

Unit 3 — Design and Architecture

- Software design transforms requirements into a structure that can be implemented.
- Modularity divides a system into manageable components. High cohesion and low coupling are desirable design properties.
- Architectural styles include layered architecture, client-server architecture, pipe-and-filter, and repository-based architecture.
- UML diagrams such as class, sequence, activity, use-case, and state diagrams communicate different aspects of a system.
- Design principles such as abstraction, encapsulation, separation of concerns, and information hiding improve maintainability.

Unit 4 — Testing

- Software testing evaluates whether a system behaves as expected and helps reveal defects.

- Unit testing checks individual components. Integration testing checks interactions among components. System testing evaluates the complete system. Acceptance testing checks whether the system satisfies stakeholder requirements.
- Black-box testing derives tests from externally observable behavior. White-box testing uses knowledge of internal structure.
- Equivalence partitioning divides input data into classes expected to behave similarly. Boundary-value analysis focuses on values at and around input boundaries.
- Test-driven development (TDD) follows a cycle commonly summarized as red, green, refactor: write a failing test, implement enough to pass, then improve the design.

Unit 5 — Project Management and Maintenance

- Software project planning estimates effort, cost, schedule, resources, and risks.
- Risk management includes identifying risks, analyzing probability and impact, planning responses, and monitoring risks.
- Software maintenance includes corrective, adaptive, perfective, and preventive maintenance.
- Configuration management controls changes to software artifacts and maintains consistent versions.
- Software quality attributes include reliability, maintainability, usability, efficiency, portability, and security.

Quick Revision Checklist

- Know the key definitions and distinguish closely related concepts.
- Practice explaining each concept in your own words before using it in an exam.
- For numerical or algorithmic topics, work through at least one example by hand.
- For database and software-engineering topics, connect definitions to practical examples.

CampusAI RAG metadata suggestion: department=CSE, semester=5, subject=Software Engineering